

Sistema de Gestión de Citas y Control de Pacientes para Centros de Salud de Cartago

Saray Foronda Restrepo

Corporación de Estudios Tecnológicos del Norte del Valle

Ingeniería de Sistemas

Cartago

23 de septiembre de 2026

Sistema de Gestión de Citas y Control de Pacientes para Centros de Salud de Cartago

Saray Foronda Restrepo

Informe de avance del proyecto – Primer corte (30 %)

Asignatura: Bases de Datos II

Profesor

Jhon James Cano Sánchez

Corporación de Estudios Tecnológicos del Norte del Valle

Ingeniería de Sistemas

Cartago

23 de septiembre de 2026

Contenido

Introducción.....	4
Objetivos del Sistema.....	4
Modelo Entidad-Relación.....	5
Descripción de las Entidades.....	6
Relaciones.....	7
Restricciones de Integridad.....	8
Normalización.....	8
Esquema Relacional Implementado.....	9
Justificación de los Tipos de Datos.....	11
Procedimientos Almacenados.....	12
sp_registrar_paciente.....	12
sp_agendar_cita.....	14
sp_cancelar_cita.....	15
sp_reporte_atenciones_centro.....	16
sp_acumular_citas_paciente.....	17
Triggers Implementados.....	18
trg_validar_cita.....	19
trg_generar_recordatorio.....	20
Evidencia del Entorno Funcionando.....	21
Conclusiones y Trabajo Futuro.....	24
Conclusiones.....	24
Trabajo Futuro.....	24
Referencias.....	26

Introducción

En el municipio de Cartago, Valle del Cauca, una parte importante de la población enfrenta necesidades básicas insatisfechas y un bajo nivel de acceso a los servicios de salud. Uno de los síntomas más visibles de esta situación es la falta de automatización en los procesos administrativos de los centros de salud: con frecuencia una persona pierde un día completo haciendo fila para agendar una cita, y quienes viven fuera del casco urbano se ven obligados a desplazarse únicamente para realizar este trámite.

Automatizar parte de estos procesos permitiría reducir la aglomeración en los puntos de atención y liberar al personal para que se enfoque en quienes no pueden acceder a medios digitales, como los adultos mayores con poco conocimiento tecnológico o las personas sin dispositivos electrónicos. Con esa motivación se plantea el presente proyecto: una base de datos relacional que soporta la gestión de pacientes, médicos, citas, historial médico básico y recordatorios automáticos para los centros de salud del municipio.

Este documento presenta el avance correspondiente al primer corte. Se describe el modelo entidad-relación, el esquema relacional implementado en MariaDB dentro de un entorno Docker, los procedimientos almacenados y los triggers desarrollados, junto con la evidencia de su funcionamiento, siguiendo la guía de avance del curso (Cano Sánchez, 2026) y las normas APA séptima edición (American Psychological Association, 2020; Barahona Gómez y Llano Castaño, 2021).

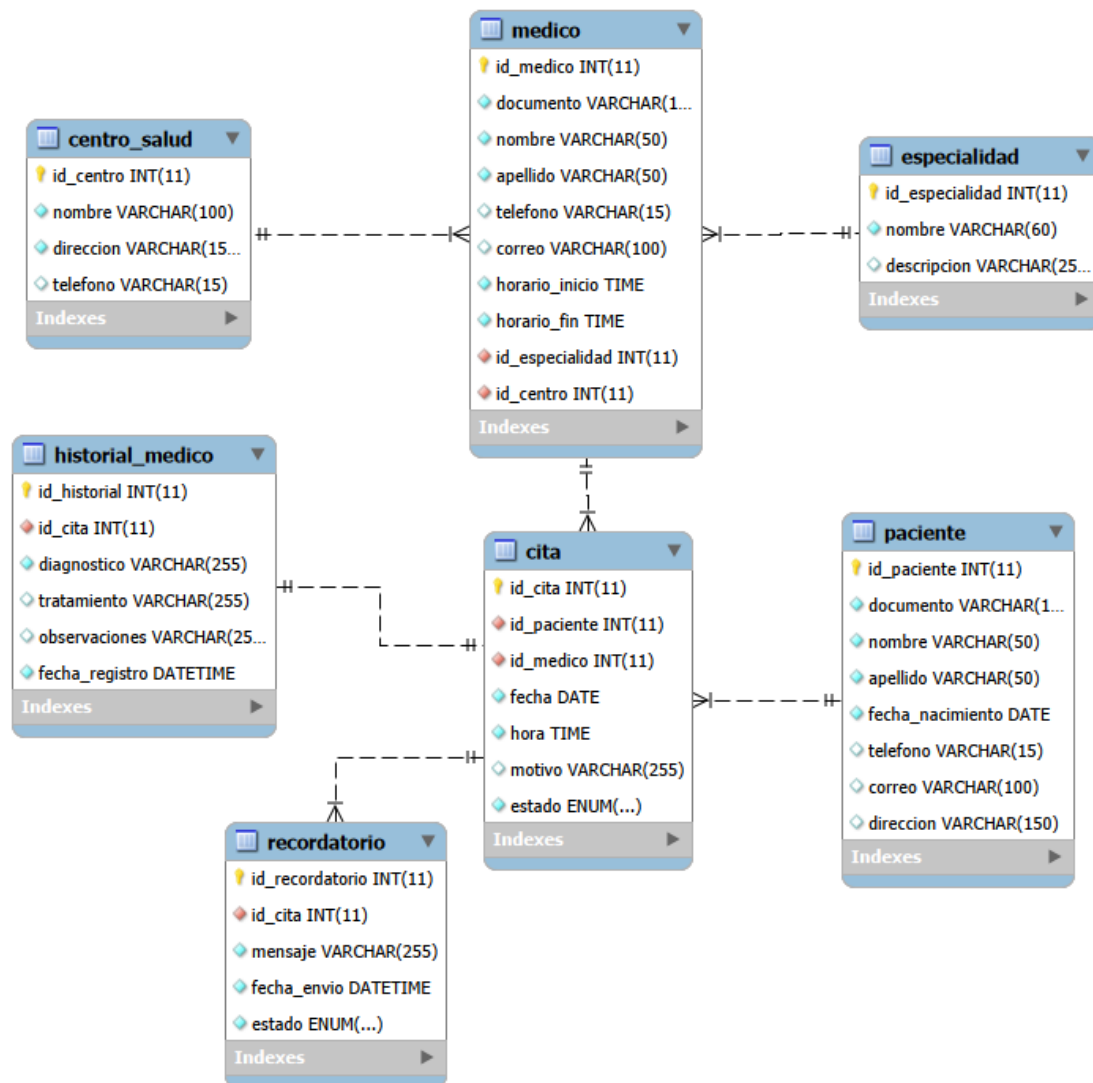
Objetivos del Sistema

1. Centralizar la información de pacientes, médicos, especialidades y centros de salud de Cartago en una base de datos normalizada que evite la duplicación de datos sensibles, como el historial médico.

2. Automatizar las operaciones frecuentes de registro de pacientes, agendamiento y cancelación de citas mediante procedimientos almacenados, reduciendo el margen de error humano.
3. Garantizar la consistencia de la agenda médica y generar recordatorios automáticos de citas mediante triggers, contribuyendo a reducir el ausentismo y ofreciendo reportes de atenciones por centro de salud.

Modelo Entidad-Relación

El modelo se diseñó a partir de las entidades principales definidas en la propuesta del proyecto: pacientes, personal médico, citas, historial médico y centros de salud. A estas se sumaron las entidades especialidad, que evita repetir el nombre de la especialidad en cada médico, y recordatorio, que almacena las notificaciones generadas automáticamente. El diagrama se generó en MySQL Workbench mediante ingeniería inversa sobre la base de datos implementada, de modo que refleja exactamente la estructura existente.

Figura 1*Modelo entidad-relación de la base de datos proyecto_citas**Nota.* Elaboración propia en MySQL Workbench.

Descripción de las Entidades

La Tabla 1 resume el propósito de cada entidad. Sus atributos se observan en la Figura 1, y el detalle de tipos de datos y restricciones de cada columna se presenta en el apartado del esquema relacional.

Tabla 1*Entidades del modelo*

Entidad	Descripción
centro_salud	Centros de salud del municipio donde se prestan las atenciones.
especialidad	Especialidades médicas ofrecidas.
medico	Personal médico, con su especialidad, el centro donde trabaja y su horario de atención.
paciente	Personas que solicitan atención médica.
cita	Citas agendadas entre un paciente y un médico, con su estado.
historial_medico	Diagnóstico, tratamiento y observaciones de cada cita atendida.
recordatorio	Notificaciones que se envían al paciente un día antes de su cita.

Relaciones

Las relaciones del modelo se presentan en la Tabla 2. Como todas las entidades tienen una llave primaria propia de tipo autoincremental, las seis relaciones son no identificativas.

La relación entre cita e historial_medico es uno a uno porque la columna id_cita del historial tiene la restricción UNIQUE, lo que impide que una cita tenga dos historiales.

Tabla 2*Relaciones entre las entidades*

Entidad padre	Entidad hija	Cardinalidad	Tipo	Significado
centro_salud	medico	1:N	No identificativa	Un centro tiene muchos médicos; cada médico trabaja en un solo centro.
especialidad	medico	1:N	No identificativa	Una especialidad agrupa a muchos médicos.
paciente	cita	1:N	No identificativa	Un paciente puede tener muchas citas.
medico	cita	1:N	No identificativa	Un médico atiende muchas citas.

Entidad padre	Entidad hija	Cardinalidad	Tipo	Significado
cita	historial_medico	1:1	No identificativa	Cada cita atendida genera un único historial.
cita	recordatorio	1:N	No identificativa	Una cita puede tener uno o varios recordatorios.

Restricciones de Integridad

La Tabla 3 resume las restricciones de integridad aplicadas en el esquema.

Tabla 3

Restricciones de integridad aplicadas

Restricción	Dónde se aplica
PRIMARY KEY	Columna id_ de cada tabla, con AUTO_INCREMENT.
FOREIGN KEY	medico (id_especialidad, id_centro), cita (id_paciente, id_medico), historial_medico (id_cita) y recordatorio (id_cita).
UNIQUE	documento de paciente y médico; nombre de centro y especialidad; id_cita en historial_medico (relación 1:1 con cita).
NOT NULL	nombres, documentos, fechas, horarios y llaves foráneas.
ENUM y DEFAULT	estado en cita (Programada por defecto) y en recordatorio (Pendiente por defecto).

Normalización

El modelo cumple la primera forma normal porque todos los atributos son atómicos: el nombre y el apellido se almacenan por separado y no existen listas ni grupos repetidos dentro de una columna. Cumple la segunda forma normal porque todas las tablas tienen una llave primaria simple, por lo que ningún atributo depende solo de una parte de la llave.

Para alcanzar la tercera forma normal se evitaron las dependencias transitivas. La especialidad no se guarda como texto dentro de la tabla de médicos, sino en su propia tabla, y se referencia mediante llave foránea; así el nombre de una especialidad se escribe una sola vez

aunque la tengan varios médicos. Del mismo modo, la cita no almacena el centro de salud: como cada médico trabaja en un único centro, el centro de una cita se obtiene a través del médico, y guardarlo también en la cita duplicaría un dato que depende de otra columna que no es la llave.

Esquema Relacional Implementado

El esquema se implementó en MariaDB 10.8 mediante el script `esquema.sql`, que crea la base de datos `proyecto_citas` y sus siete tablas en orden de dependencia: primero las que no tienen llaves foráneas (`centro_salud`, `especialidad` y `paciente`) y después las que dependen de ellas. La Tabla 4 detalla las columnas de cada tabla.

Tabla 4

Esquema relacional de la base de datos proyecto_citas

Tabla	Columna	Tipo	Restricciones
centro_salud	id_centro	INT	PRIMARY KEY, AUTO_INCREMENT
	nombre	VARCHAR(100)	NOT NULL, UNIQUE
	direccion	VARCHAR(150)	NOT NULL
	telefono	VARCHAR(15)	Opcional
especialidad	id_especialidad	INT	PRIMARY KEY, AUTO_INCREMENT
	nombre	VARCHAR(60)	NOT NULL, UNIQUE
	descripcion	VARCHAR(255)	Opcional
medico	id_medico	INT	PRIMARY KEY, AUTO_INCREMENT
	documento	VARCHAR(15)	NOT NULL, UNIQUE
	nombre	VARCHAR(50)	NOT NULL
	apellido	VARCHAR(50)	NOT NULL
	telefono	VARCHAR(15)	Opcional
	correo	VARCHAR(100)	Opcional
	horario_inicio	TIME	NOT NULL
	horario_fin	TIME	NOT NULL

Tabla	Columna	Tipo	Restricciones
paciente	id_especialidad	INT	NOT NULL, FOREIGN KEY → especialidad
	id_centro	INT	NOT NULL, FOREIGN KEY → centro_salud
	id_paciente	INT	PRIMARY KEY, AUTO_INCREMENT
	documento	VARCHAR(15)	NOT NULL, UNIQUE
	nombre	VARCHAR(50)	NOT NULL
	apellido	VARCHAR(50)	NOT NULL
	fecha_nacimiento	DATE	NOT NULL
	telefono	VARCHAR(15)	Opcional
	correo	VARCHAR(100)	Opcional
	direccion	VARCHAR(150)	Opcional
cita	id_cita	INT	PRIMARY KEY, AUTO_INCREMENT
	id_paciente	INT	NOT NULL, FOREIGN KEY → paciente
	id_medico	INT	NOT NULL, FOREIGN KEY → medico
	fecha	DATE	NOT NULL
	hora	TIME	NOT NULL
	motivo	VARCHAR(255)	Opcional
	estado	ENUM('Programada', 'Atendida', 'Cancelada', 'No asistio')	NOT NULL, DEFAULT 'Programada'
historial_medico	id_historial	INT	PRIMARY KEY, AUTO_INCREMENT
	id_cita	INT	NOT NULL, UNIQUE, FOREIGN KEY → cita
	diagnostico	VARCHAR(255)	NOT NULL
	tratamiento	VARCHAR(255)	Opcional
	observaciones	VARCHAR(255)	Opcional
	fecha_registro	DATETIME	NOT NULL

Tabla	Columna	Tipo	Restricciones
recordatorio	id_recordatorio	INT	PRIMARY KEY, AUTO_INCREMENT
	id_cita	INT	NOT NULL, FOREIGN KEY → cita
	mensaje	VARCHAR(255)	NOT NULL
	fecha_envio	DATETIME	NOT NULL
	estado	ENUM('Pendiente', 'Enviado', 'Cancelado')	NOT NULL, DEFAULT 'Pendiente'

Justificación de los Tipos de Datos

INT se utilizó en todas las llaves primarias y foráneas, junto con **AUTO_INCREMENT** para que los identificadores se asignen automáticamente; su rango es suficiente para el volumen esperado.

VARCHAR se eligió para los textos de longitud variable, ajustando el tamaño máximo a cada dato: 50 caracteres para nombres, 100 para correos y 150 para direcciones. Los documentos de identidad y los teléfonos también se almacenan como **VARCHAR** y no como números, porque pueden comenzar con cero, no se realizan operaciones aritméticas con ellos y así se evita perder dígitos.

DATE, **TIME** y **DATETIME** se usaron según la información requerida: **DATE** para la fecha de nacimiento y la fecha de la cita, **TIME** para la hora de la cita y los horarios de los médicos, y **DATETIME** cuando se necesita el momento exacto, como en la fecha de registro del historial o la fecha de envío de un recordatorio. Separar la fecha y la hora de la cita facilita las consultas por día y la validación de horarios que realiza el trigger.

ENUM se empleó en los estados de la cita y del recordatorio porque solo admiten un conjunto cerrado de valores, lo que evita estados mal escritos.

Los datos de prueba se cargaron con el script `datos_prueba.sql`, que inserta como mínimo diez registros por tabla. Los nombres de centros, médicos y pacientes son ficticios.

Procedimientos Almacenados

Se desarrollaron cinco procedimientos almacenados, contenidos en el script `procedimientos.sql` (MariaDB Foundation, s.f.-a), que cubren las operaciones planteadas en la propuesta del proyecto y usan los tres modos de parámetro. La Tabla 5 resume sus parámetros.

Tabla 5

Parámetros de los procedimientos almacenados

Procedimiento	IN	OUT / INOUT
<code>sp_registrar_paciente</code>	Documento, nombre, apellido, fecha de nacimiento, teléfono, correo y dirección del paciente.	OUT <code>p_id_paciente</code> : id asignado al paciente.
<code>sp_agendar_cita</code>	Paciente, médico, fecha, hora y motivo de la cita.	OUT <code>p_id_cita</code> : id de la cita creada.
<code>sp_cancelar_cita</code>	<code>p_id_cita</code> : cita a cancelar.	No tiene.
<code>sp_reporte_atenciones_centro</code>	<code>p_id_centro</code> : centro a consultar.	OUT <code>p_total_atendidas</code> : total de citas atendidas.
<code>sp_acumular_citas_paciente</code>	<code>p_id_paciente</code> : paciente cuyas citas se suman.	INOUT <code>p_total</code> : entra con el acumulado previo y sale con el nuevo total.

`sp_registrar_paciente`

Registra un paciente nuevo a partir de sus datos personales y devuelve el identificador que el sistema le asignó.

Código SQL:

```
DELIMITER $$

CREATE PROCEDURE sp_registrar_paciente(
    IN p_documento VARCHAR(15),
    IN p_nombre VARCHAR(50),
    IN p_apellido VARCHAR(50),
```

```

    IN p_fecha_nacimiento DATE,
    IN p_telefono VARCHAR(15),
    IN p_correo VARCHAR(100),
    IN p_direccion VARCHAR(150),
    OUT p_id_paciente INT
)
BEGIN
    INSERT INTO paciente (documento, nombre, apellido, fecha_nacimiento,
telefono, correo, direccion)
    VALUES (p_documento, p_nombre, p_apellido, p_fecha_nacimiento,
p_telefono, p_correo, p_direccion);

    SELECT id_paciente INTO p_id_paciente
    FROM paciente
    WHERE documento = p_documento;
END$$

DELIMITER ;

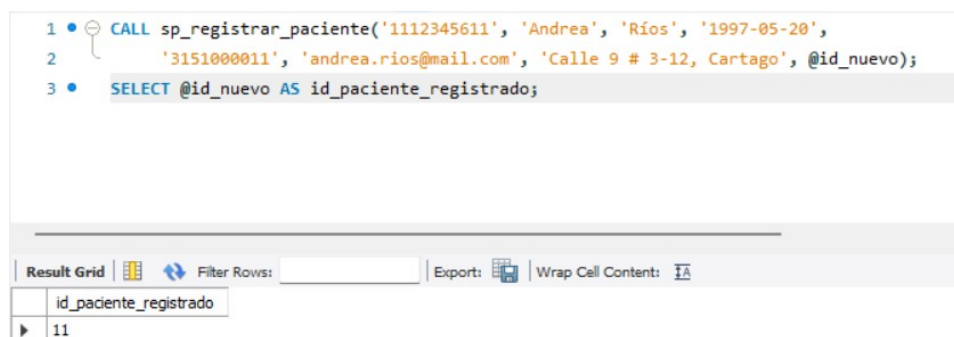
```

Problema que resuelve: estandariza el registro de pacientes en una sola operación y entrega de inmediato el identificador asignado, que se necesita para agendarle una cita. El identificador se recupera buscando por el documento, que es único.

Ejemplo de ejecución: se registró a la paciente Andrea Ríos y el procedimiento devolvió en la variable @id_nuevo el identificador 11 (Figura 2).

Figura 2

Ejecución de *sp_registrar_paciente*



Nota. Captura propia en MySQL Workbench.

sp_agendar_cita

Agenda una cita entre un paciente y un médico en una fecha y hora dadas. La cita queda en estado Programada gracias al valor por defecto de la columna estado, y el procedimiento devuelve el identificador de la cita creada.

Código SQL:

```
DELIMITER $$

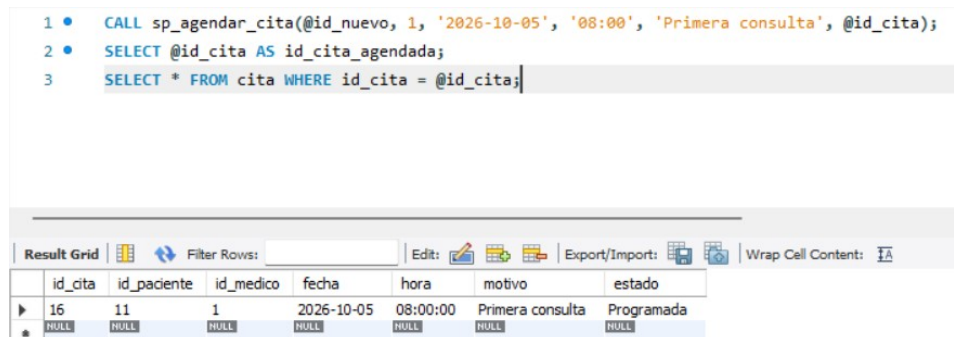
CREATE PROCEDURE sp_agendar_cita(
    IN p_id_paciente INT,
    IN p_id_medico INT,
    IN p_fecha DATE,
    IN p_hora TIME,
    IN p_motivo VARCHAR(255),
    OUT p_id_cita INT
)
BEGIN
    INSERT INTO cita (id_paciente, id_medico, fecha, hora, motivo)
    VALUES (p_id_paciente, p_id_medico, p_fecha, p_hora, p_motivo);

    SELECT MAX(id_cita) INTO p_id_cita
    FROM cita;
END$$

DELIMITER ;
```

Problema que resuelve: centraliza el agendamiento, que es la operación más frecuente del sistema. Al insertar la cita se activan los triggers descritos más adelante, que validan la disponibilidad del médico y generan el recordatorio automático.

Ejemplo de ejecución: se usó como paciente la variable @id_nuevo obtenida en el procedimiento anterior, lo que simula el flujo real: primero se registra al paciente y luego se le agenda la cita. El resultado fue la cita 16 en estado Programada (Figura 3).

Figura 3*Ejecución de sp_agendar_cita*


The screenshot shows the MySQL Workbench interface. The top pane contains three SQL statements:

- 1 • `CALL sp_agendar_cita(@id_nuevo, 1, '2026-10-05', '08:00', 'Primera consulta', @id_cita);`
- 2 • `SELECT @id_cita AS id_cita_agendada;`
- 3 • `SELECT * FROM cita WHERE id_cita = @id_cita;`

The bottom pane displays the 'Result Grid' for the third query. It shows a table with 7 columns: `id_cita`, `id_paciente`, `id_medico`, `fecha`, `hora`, `motivo`, and `estado`. The first row contains the values: 16, 11, 1, 2026-10-05, 08:00:00, Primera consulta, and Programada. Below this row, there is a row of null values for each column, and a row starting with an asterisk.

id_cita	id_paciente	id_medico	fecha	hora	motivo	estado
16	11	1	2026-10-05	08:00:00	Primera consulta	Programada
NULL	NULL	NULL	NULL	NULL	NULL	NULL
*						

Nota. Captura propia en MySQL Workbench.

sp_cancelar_cita

Cancela una cita cambiando su estado a Cancelada y, en la misma operación, marca como Cancelado el recordatorio que tuviera pendiente.

Código SQL:

```
DELIMITER $$

CREATE PROCEDURE sp_cancelar_cita(
    IN p_id_cita INT
)
BEGIN
    UPDATE cita
    SET estado = 'Cancelada'
    WHERE id_cita = p_id_cita;

    UPDATE recordatorio
    SET estado = 'Cancelado'
    WHERE id_cita = p_id_cita AND estado = 'Pendiente';
END$$

DELIMITER ;
```

Problema que resuelve: evita que al cancelar una cita quede un recordatorio pendiente que se enviaría al paciente por error. Al actualizar las dos tablas en un solo procedimiento, la información queda consistente.

Ejemplo de ejecución: la cita 14 estaba en estado Programada y su recordatorio en estado Pendiente. Después de ejecutar `CALL sp_cancelar_cita(14)`, la cita quedó Cancelada y el recordatorio Cancelado (Figura 4).

Figura 4

Resultado de `sp_cancelar_cita` sobre la cita 14



The screenshot shows a MySQL Workbench interface. At the top, a SQL query is displayed in a text editor:

```

1  SELECT c.id_cita, c.estado AS estado_cita,
2      r.id_recordatorio, r.estado AS estado_recordatorio
3  FROM cita c
4  INNER JOIN recordatorio r ON r.id_cita = c.id_cita
5  WHERE c.id_cita = 14;

```

Below the query editor, the 'Result Grid' tab is active, showing the results of the query. The grid has four columns: `id_cita`, `estado_cita`, `id_recordatorio`, and `estado_recordatorio`. There is one row of data:

id_cita	estado_cita	id_recordatorio	estado_recordatorio
14	Cancelada	9	Cancelado

Nota. Captura propia en MySQL Workbench.

sp_reporte_atenciones_centro

Genera el reporte de atenciones de un centro de salud: lista las citas atendidas con el paciente, el médico, la especialidad y el diagnóstico, y devuelve el total de atenciones.

Código SQL:

```

DELIMITER $$

CREATE PROCEDURE sp_reporte_atenciones_centro(
    IN p_id_centro INT,
    OUT p_total_atendidas INT
)
BEGIN
    SELECT cs.nombre AS centro,
           c.fecha,
           p.nombre AS paciente, p.apellido AS apellido_paciente,
           m.nombre AS medico, m.apellido AS apellido_medico,
           e.nombre AS especialidad,
           h.diagnostico
    FROM cita c
    INNER JOIN medico m ON c.id_medico = m.id_medico
    INNER JOIN centro_salud cs ON m.id_centro = cs.id_centro
    INNER JOIN especialidad e ON m.id_especialidad = e.id_especialidad

```

```

INNER JOIN paciente p ON c.id_paciente = p.id_paciente
INNER JOIN historial_medico h ON h.id_cita = c.id_cita
WHERE cs.id_centro = p_id_centro AND c.estado = 'Atendida';

SELECT COUNT(*) INTO p_total_atendidas
FROM cita c
INNER JOIN medico m ON c.id_medico = m.id_medico
WHERE m.id_centro = p_id_centro AND c.estado = 'Atendida';
END$$

DELIMITER ;

```

Problema que resuelve: ofrece al centro de salud información útil para la toma de decisiones, como qué especialidades tienen más demanda. Es además la consulta compleja del proyecto, porque une cinco tablas mediante INNER JOIN.

Ejemplo de ejecución: para el Centro de Salud Santa Ana (id 3), el procedimiento listó la atención de cardiología realizada al paciente Luis Cardona, con diagnóstico de angina estable (Figura 5), y devolvió en @total_centro un total de 1 atención.

Figura 5

Listado de atenciones del centro 3 obtenido con INNER JOIN

```
1 • CALL sp_reporte_atenciones_centro(3, @total_centro);
```

Result Grid							
Filter Rows:		Export:		Wrap Cell Content:			
centro	fecha	paciente	apellido_paciente	medico	apellido_medico	especialidad	diagnostico
Centro de Salud Santa Ana	2026-08-10	Luis	Cardona	Carlos	Ramírez	Cardiología	Angina estable

Nota. Captura propia en MySQL Workbench.

sp_acumular_citas_paciente

Suma al valor recibido la cantidad de citas de un paciente y devuelve el total actualizado. Llamándolo varias veces con la misma variable se obtiene el total de citas de un grupo de pacientes, por ejemplo, de un grupo familiar.

Código SQL:

```
DELIMITER $$
```

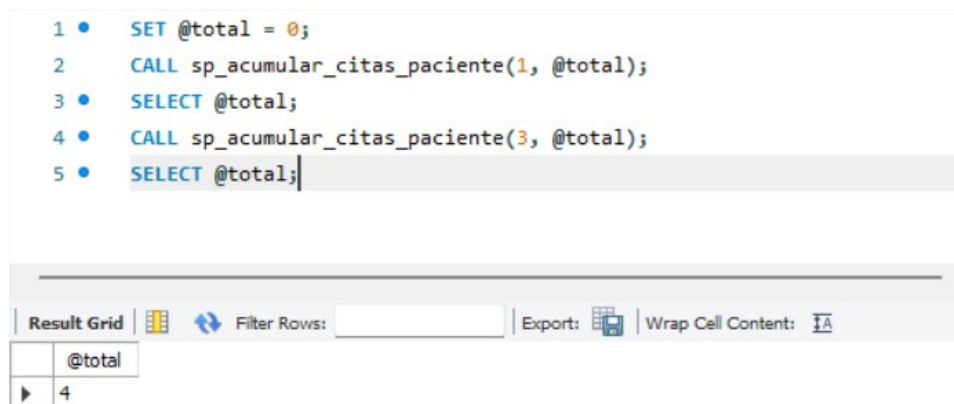
```
CREATE PROCEDURE sp_acumular_citas_paciente(  
    IN p_id_paciente INT,  
    INOUT p_total INT  
)  
BEGIN  
    SELECT p_total + COUNT(*) INTO p_total  
    FROM cita  
    WHERE id_paciente = p_id_paciente;  
END$$  
  
DELIMITER ;
```

Problema que resuelve: permite obtener el total de citas de un grupo de pacientes, reutilizando el resultado de cada llamada como entrada de la siguiente.

Ejemplo de ejecución: la variable @total inició en 0. Tras la primera llamada tomó el valor 2, correspondiente a las citas del paciente 1, y tras la segunda llegó a 4, porque sumó las dos citas del paciente 3 al valor que ya traía (Figura 6).

Figura 6

Ejecución de sp_acumular_citas_paciente con parámetro INOUT



```
1 • SET @total = 0;  
2 • CALL sp_acumular_citas_paciente(1, @total);  
3 • SELECT @total;  
4 • CALL sp_acumular_citas_paciente(3, @total);  
5 • SELECT @total;
```

@total
4

Nota. Captura propia en MySQL Workbench.

Triggers Implementados

Se implementaron dos triggers sobre la tabla cita, contenidos en el script `triggers.sql` (MariaDB Foundation, s.f.-b): uno de validación (BEFORE INSERT) y uno de automatización (AFTER INSERT).

trg_validar_cita

Se ejecuta antes de insertar una cita y realiza dos validaciones. La primera cuenta las citas activas del mismo médico en la misma fecha y hora, sin tener en cuenta las canceladas; si ya existe alguna, detiene la inserción. La segunda compara la hora de la cita con el horario de atención del médico y detiene la inserción si la hora queda por fuera. En ambos casos se utiliza SIGNAL SQLSTATE '45000' para generar un error con un mensaje propio.

Código SQL:

```
DELIMITER $$

CREATE TRIGGER trg_validar_cita
BEFORE INSERT ON cita
FOR EACH ROW
BEGIN
    -- a) Médico ocupado (no cuentan las citas canceladas)
    IF (SELECT COUNT(*) FROM cita
        WHERE id_medico = NEW.id_medico
        AND fecha = NEW.fecha
        AND hora = NEW.hora
        AND estado <> 'Cancelada') > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'El medico ya tiene una cita asignada en esa fecha
y hora';
    END IF;

    -- b) Hora fuera del horario del médico
    IF NEW.hora < (SELECT horario_inicio FROM medico WHERE id_medico =
NEW.id_medico)
    OR NEW.hora >= (SELECT horario_fin FROM medico WHERE id_medico =
NEW.id_medico) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La hora de la cita esta fuera del horario de
atencion del medico';
    END IF;
END$$

DELIMITER ;
```

Utilidad: garantiza la consistencia de la agenda médica, porque impide que a un médico se le asignen dos pacientes a la misma hora o citas fuera de su jornada. Se definió como BEFORE porque la validación debe ocurrir antes de que el registro se guarde, para poder rechazarlo.

Ejemplo de funcionamiento: el médico 1 atiende de 07:00 a 13:00 y ya tiene una cita el 5 de octubre de 2026 a las 08:00. Al intentar agendar otra cita en ese mismo horario, el trigger la rechazó con el mensaje correspondiente (Figura 7). Al intentar agendarla a las 15:00, fuera de su horario, también la rechazó (Figura 8).

Figura 7

Rechazo de una cita por médico ocupado



77 00:11:07 CALL sp_agendar_cita(2, 1, '2026-10-05', '08:00', 'Prueba choque', @x) Error Code: 1644. El medico ya tiene una cita asignada en esa fecha y hora 0.015 sec

Nota. Captura propia del panel Output de MySQL Workbench.

Figura 8

Rechazo de una cita fuera del horario del médico



78 00:13:02 CALL sp_agendar_cita(2, 1, '2026-10-05', '15:00', 'Prueba horario', @x) Error Code: 1644. La hora de la cita esta fuera del horario de atencion del medico 0.015 sec

Nota. Captura propia del panel Output de MySQL Workbench.

trg_generar_recordatorio

Se ejecuta después de insertar una cita y crea automáticamente su recordatorio en la tabla recordatorio. El mensaje se construye con CONCAT a partir de la fecha y la hora de la cita, y la fecha de envío se calcula con DATE_SUB restando un día a la fecha y hora de la cita.

Código SQL:

```
DELIMITER $$

CREATE TRIGGER trg_generar_recordatorio
AFTER INSERT ON cita
FOR EACH ROW
BEGIN
    INSERT INTO recordatorio (id_cita, mensaje, fecha_envio, estado)
    VALUES (
        NEW.id_cita,
        CONCAT('Recuerde su cita del ', NEW.fecha, ' a las ', NEW.hora),
        DATE_SUB(CONCAT(NEW.fecha, ' ', NEW.hora), INTERVAL 1 DAY),
        'Pendiente'
    );
END$$

DELIMITER ;
```

Utilidad: cumple el alcance de recordatorios automáticos planteado en la propuesta.

Ninguna persona tiene que registrar el recordatorio a mano, lo que elimina el riesgo de olvidarlo y contribuye a reducir el ausentismo. Se definió como AFTER porque necesita el identificador de la cita, que solo existe una vez insertado el registro.

Ejemplo de funcionamiento: se agendó una cita válida para el 5 de octubre de 2026 a las 09:00. Sin ninguna inserción manual, el trigger creó el recordatorio 11 para la cita 17, con fecha de envío 4 de octubre de 2026 a las 09:00 y estado Pendiente (Figura 9).

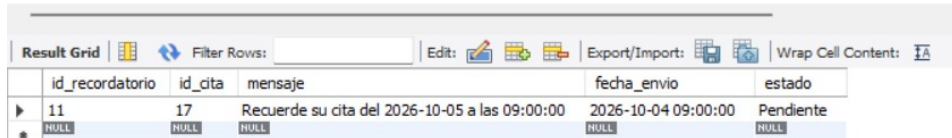
Figura 9

Recordatorio generado automáticamente por trg_generar_recordatorio

```

1 CALL sp_agendar_cita(4, 1, '2026-10-05', '09:00', 'Control general', @id_cita_nueva);
2 • SELECT @id_cita_nueva;
3 • SELECT * FROM recordatorio WHERE id_cita = @id_cita_nueva;

```



id_recordatorio	id_cita	mensaje	fecha_envio	estado
11	17	Recuerde su cita del 2026-10-05 a las 09:00:00	2026-10-04 09:00:00	Pendiente

Nota. Captura propia en MySQL Workbench.

Evidencia del Entorno Funcionando

El proyecto se ejecuta sobre WSL con Ubuntu (Microsoft, s.f.) y Docker, y la base de datos se administra con MySQL Workbench (Oracle Corporation, s.f.). El archivo docker-compose.yml del repositorio define tres servicios comunicados por una red interna: PHP 8.2 con Apache (puerto 8080), MariaDB 10.8 (puerto 3306) y phpMyAdmin (puerto 8081), con un volumen persistente para los datos (Docker Inc., s.f.). En el equipo de desarrollo, por conflicto de puertos, la base de datos se desplegó en un entorno Docker existente con la misma arquitectura: sge_php (puerto 8082), sge_mariadb (puerto 3307) y sge_phpmyadmin (puerto 8083), como

muestra la Figura 10. Los scripts SQL son independientes del contenedor, por lo que la base puede recrearse en cualquiera de los dos entornos.

Figura 10

Contenedores activos mostrados con el comando docker ps

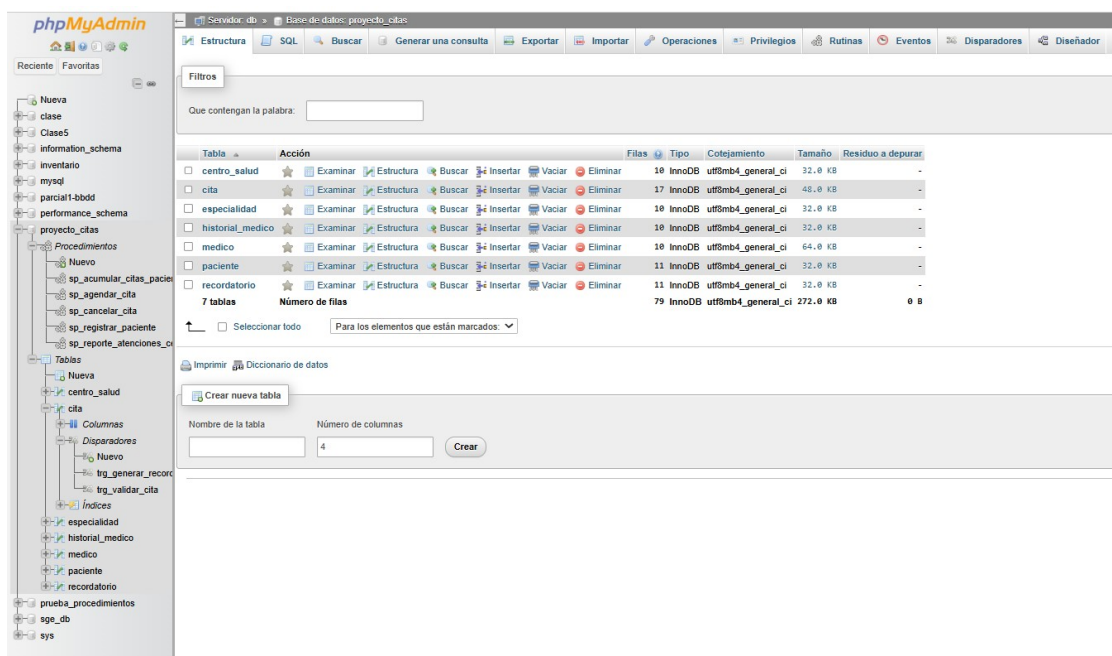
```
sarayforonda@HPSaray:/mnt/c/Base_Datos_II/proyecto-BDII$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
33b2298bbf53	softwaregestionempresarial-app	sg_e_php	"docker-php-entrypoi..."	4 weeks ago	Up 29 hours	0.0.0.0:8082->80/tcp
0cdd3a094a72	phpmyadmin/phpmyadmin:latest	sg_phpmyadmin	"/docker-entrypoint.s..."	4 weeks ago	Up 29 hours	0.0.0.0:8083->80/tcp
b74efa4fc109	mariadb:10.8	sg_mariadb	"docker-entrypoint.s..."	4 weeks ago	Up 29 hours	0.0.0.0:3307->3306/tcp

Nota. Captura propia de la terminal de WSL.

Figura 11

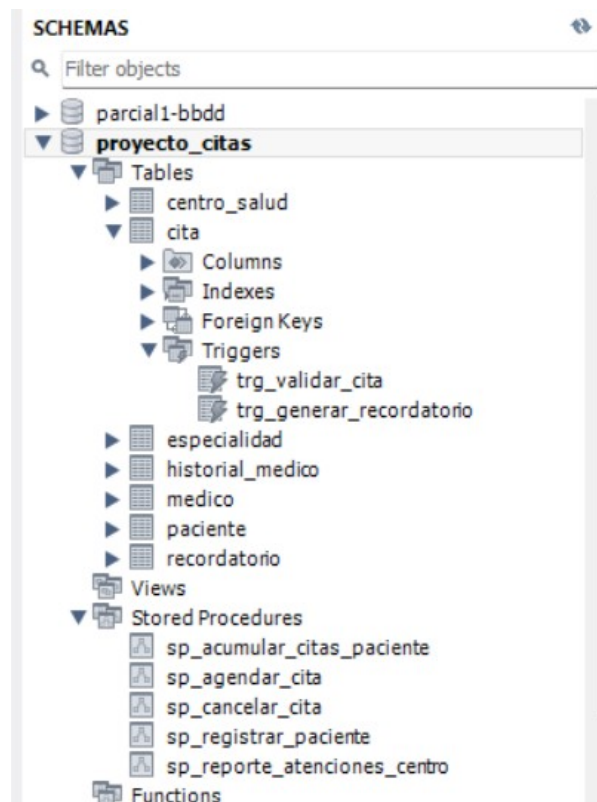
Acceso a phpMyAdmin con la base de datos proyecto_citas



Nota. Captura propia del navegador (<http://localhost:8083>). En el panel izquierdo se observan las siete tablas, los cinco procedimientos almacenados y los dos triggers de la tabla cita.

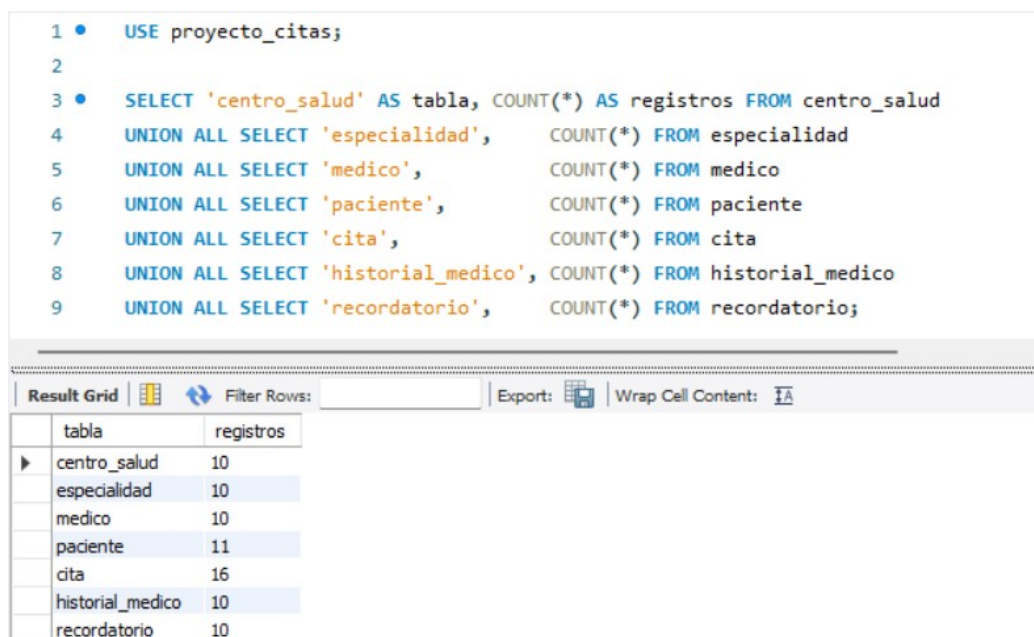
Figura 12

Conexión desde MySQL Workbench con las tablas del proyecto



Nota. Captura propia del panel Schemas de MySQL Workbench: tablas, triggers de la tabla cita y procedimientos almacenados del esquema proyecto_citas.

La Figura 13 comprueba que cada tabla tiene al menos diez registros; paciente (11) y cita (16) incluyen los que agregaron los procedimientos durante las pruebas. La consulta compleja con JOIN corresponde a la Figura 5.

Figura 13*Cantidad de registros por tabla*


```

1 • USE proyecto_citas;
2
3 • SELECT 'centro_salud' AS tabla, COUNT(*) AS registros FROM centro_salud
4 UNION ALL SELECT 'especialidad', COUNT(*) FROM especialidad
5 UNION ALL SELECT 'medico', COUNT(*) FROM medico
6 UNION ALL SELECT 'paciente', COUNT(*) FROM paciente
7 UNION ALL SELECT 'cita', COUNT(*) FROM cita
8 UNION ALL SELECT 'historial_medico', COUNT(*) FROM historial_medico
9 UNION ALL SELECT 'recordatorio', COUNT(*) FROM recordatorio;

```

tabla	registros
centro_salud	10
especialidad	10
medico	10
paciente	11
cita	16
historial_medico	10
recordatorio	10

Nota. Captura propia en MySQL Workbench.

Conclusiones y Trabajo Futuro

Conclusiones

Este avance permitió construir, a partir de la propuesta del proyecto, una base de datos normalizada hasta la tercera forma normal, con restricciones de integridad que protegen la consistencia de la información y funcional en un entorno Docker reproducible.

Los cinco procedimientos almacenados cubren el registro de pacientes, el agendamiento y la cancelación de citas y los reportes por centro, y los dos triggers automatizan la validación de la agenda médica y la generación de recordatorios, que eran los objetivos planteados para este corte.

Trabajo Futuro

Para los siguientes cortes se proponen estas mejoras:

- **Seguridad:** incorporar una tabla de usuarios con roles (administrador, recepcionista, médico y paciente), crear usuarios de base de datos con privilegios limitados y mover las credenciales del docker-compose.yml a un archivo .env que no se suba al repositorio.
- **Historial médico:** crear un procedimiento que registre la atención de una cita, insertando el historial y cambiando el estado de la cita a Atendida en una sola operación.
- **Optimización:** analizar las consultas más frecuentes para crear índices adicionales y construir vistas que simplifiquen los reportes por especialidad y por período.

Referencias

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). <https://doi.org/10.1037/0000165-000>
- Barahona Gómez, A. y Llano Castaño, Y. M. (2021). *Guía normas APA séptima edición*. Corporación de Estudios Tecnológicos del Norte del Valle.
- Cano Sánchez, J. J. (2026). *Guía para el avance del proyecto – primer corte* [Material de clase]. Bases de Datos II, Corporación de Estudios Tecnológicos del Norte del Valle.
- Docker Inc. (s.f.). *Docker Compose overview*. Docker Docs. <https://docs.docker.com/compose/>
- MariaDB Foundation. (s.f.-a). *CREATE PROCEDURE*. MariaDB Knowledge Base. <https://mariadb.com/kb/en/create-procedure/>
- MariaDB Foundation. (s.f.-b). *CREATE TRIGGER*. MariaDB Knowledge Base. <https://mariadb.com/kb/en/create-trigger/>
- Microsoft. (s.f.). *How to install Linux on Windows with WSL*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/wsl/install>
- Oracle Corporation. (s.f.). *MySQL Workbench manual*. <https://dev.mysql.com/doc/workbench/en/>